Section 4.3: Building a CNN

Activation, Pooling and the Classifier Head

AI-IM Class — Lecture Notes

Section 4.2 gave us the convolutional layer. A real CNN wraps every convolution with two more steps and finishes with the MLP classifier of Section 3.4. In this section we

- describe the three stages of a typical CNN layer,
- compute max and average pooling by hand and see why pooling gives invariance to small shifts,
- assemble a complete CNN for MNIST and trace its shapes and parameters, and
- see how gradients pass back through pooling.

1 The Three Stages of a CNN Layer

Goodfellow et al. describe a typical CNN layer as three stages applied in sequence:

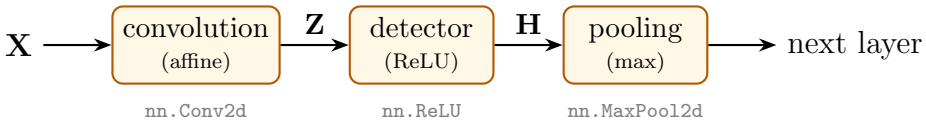


Figure 1: The three stages of a CNN layer, with the matching PyTorch modules.

1. **Convolution** computes the pre-activations $\mathbf{Z}$, as in Section 4.2. Like $\mathbf{Wh} + \mathbf{b}$ in an MLP, it is linear (affine).
2. **Detector stage.** A nonlinear activation, almost always ReLU, is applied to each entry: $\mathbf{H} = \mathrm{ReLU}(\mathbf{Z})$. Without it, stacked convolutions would collapse into one linear map, exactly as stacked linear layers do in an MLP.
3. **Pooling** replaces each small region of $\mathbf{H}$ by a summary statistic, such as its maximum. This is new, and is the topic of the next section.

Exercise 1 The detector stage

A convolution produced the feature map $\mathbf{Z}$ below. Apply ReLU.

$$\mathbf{Z} = \begin{pmatrix} 1 & -2 & 3 & 0 \\ -1 & 4 & -3 & 2 \\ 0 & 1 & 5 & -4 \\ 2 & -1 & 1 & 3 \end{pmatrix} \quad \Longrightarrow \quad \mathbf{H} = \mathrm{ReLU}(\mathbf{Z}) = \begin{pmatrix} 1 & 0 & 3 & 0 \\ 0 & 4 & 0 & 2 \\ 0 & 1 & 5 & 0 \\ 2 & 0 & 1 & 3 \end{pmatrix}$$

2 Pooling

A **pooling** layer slides a window over each feature map, just like a convolution, but instead of a weighted sum it computes a fixed summary of the window:

- **max pooling** takes the largest value (most common);
- **average pooling** takes the mean.

The usual choice is a 2×2 window with stride 2, so the windows do not overlap and each feature map shrinks to half its height and width.

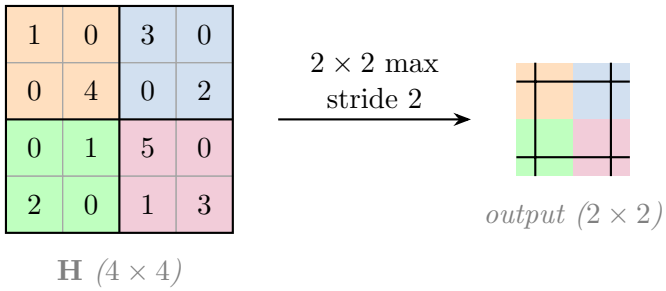


Figure 2: Max pooling with a 2×2 window and stride 2. Each coloured block of four values becomes one output value.

Exercise 2 Max and average pooling

Pool the feature map $\mathbf{H}$ from Exercise 1 (Figure 2) with a 2×2 window and stride 2.

$$\text{max pooling: } \begin{pmatrix} 4 & 3 \\ 2 & 5 \end{pmatrix}, \qquad \text{average pooling: } \begin{pmatrix} 1.25 & 1.25 \\ 0.75 & 2.25 \end{pmatrix}$$

What fraction of the values is discarded? $12/16 = 75\%$

Some facts about pooling layers:

- A pooling layer has **no parameters**. Nothing is learned.
- It acts on **each channel separately**: a (C, H, W) input gives a (C, H', W') output with the same number of channels.
- The output size follows the same formula as convolution, $n_{\text{out}} = \lfloor (n+2p-k)/s \rfloor + 1$. For $k = s = 2$ and $p = 0$ this is $\lfloor n/2 \rfloor$.
- By shrinking the feature maps it cuts the computation of all later layers, and it makes the receptive field grow much faster than stacking convolutions alone (Section 4.1).

2.1 Pooling Gives Invariance to Small Shifts

In Section 4.1 we saw that convolution is *equivariant*: shift the input and the feature map shifts with it. Pooling makes the representation approximately *invariant*: if the input moves a little, most pooled values do not change at all. The reason is simple. The maximum of a window does not care *where* in the window the largest value is.

Exercise 3 Shift the input by one pixel

A detector stage produced the 1-D output $\mathbf{h}_A$. In $\mathbf{h}_B$ the input was shifted one pixel to the right, so every value moved one place. Apply max pooling with window 3 and stride 1.

$\mathbf{h}_A$	1	5	2	0	4	1	0
max pool	5	5	4	4	4		
$\mathbf{h}_B$	0	1	5	2	0	4	1
max pool	5	5	5	4	4		

Between $\mathbf{h}_A$ and $\mathbf{h}_B$, 7 of the 7 detector values changed, but only 1 of the 5 pooled values changed.

This invariance is *local*: it holds for shifts smaller than the pooling window. Stacking several conv–pool layers extends it to larger shifts, which is what a classifier needs: the answer "digit 7" should not depend on exactly where the 7 is written.

Pooling over channels. Pooling need not be over space. If several filters each detect the same shape at a different rotation, taking the maximum *across those channels* gives a unit that responds to the shape at any of those rotations. The network can learn invariances that convolution alone does not provide (Goodfellow et al., Figure 9.9).

Pooling or stride? A convolution with stride 2 also halves the feature map, and unlike pooling it has learned weights. Many modern architectures (e.g. ResNet, Section 4.4) shrink the maps mostly with strided convolutions and use pooling only at the start and end.

3 Assembling a Complete CNN

A CNN for classification has two parts:

1. a **feature extractor**: a stack of conv–ReLU–pool blocks. Each block halves the height and width and usually doubles the number of channels. The network trades spatial detail (*where*) for richer features (*what*);
2. a **classifier head**: flatten the final feature maps into a vector and feed it to the MLP classifier of Section 3.4, ending in C logits, softmax and the cross-entropy loss.

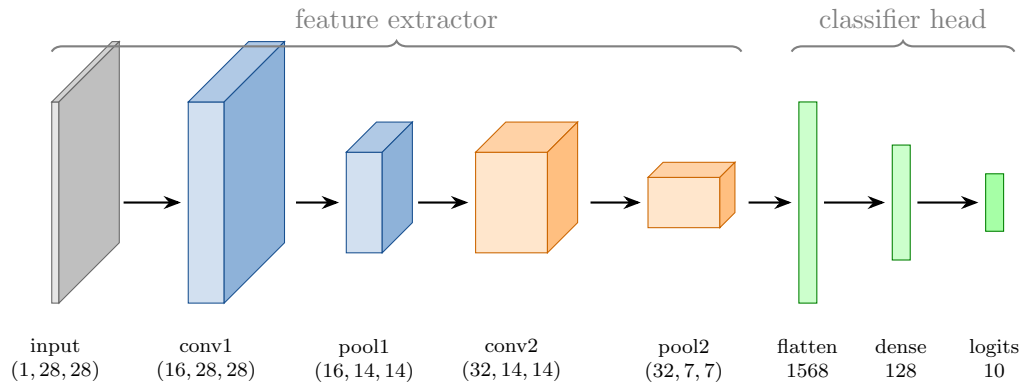


Figure 3: A small CNN for MNIST. Each conv layer is followed by ReLU (not drawn). The feature maps get smaller and deeper; the final maps are flattened and classified by an MLP.

In PyTorch the whole network is a short `nn.Sequential`:

```
model = nn.Sequential(
    nn.Conv2d(1, 16, kernel_size=3, padding=1),   # conv1
    nn.ReLU(),
    nn.MaxPool2d(2),                              # pool1
    nn.Conv2d(16, 32, kernel_size=3, padding=1),  # conv2
    nn.ReLU(),
    nn.MaxPool2d(2),                              # pool2
    nn.Flatten(),
    nn.Linear(32 * 7 * 7, 128),
    nn.ReLU(),
    nn.Linear(128, 10),                           # logits
)
```

As in Section 3.4, the model outputs logits; `nn.CrossEntropyLoss` applies softmax and cross-entropy together.

Exercise 4 Trace the network

Fill in the output shape (for one image) and the number of parameters of every layer in the model above.

Layer	Output shape	Parameters
Input	$(1, 28, 28)$	—
Conv $1 \to 16$, 3×3, p=1	$(16, 28, 28)$	$(1 \cdot 9 + 1) \cdot 16 = 160$
ReLU	$(16, 28, 28)$	0
MaxPool 2×2	$(16, 14, 14)$	0
Conv $16 \to 32$, 3×3, p=1	$(32, 14, 14)$	$(16 \cdot 9 + 1) \cdot 32 = 4\,640$
MaxPool 2×2	$(32, 7, 7)$	0
Flatten	(1568)	0
Linear $\to 128$	(128)	$(1568 + 1) \cdot 128 = 200\,832$
Linear $\to 10$	(10)	$(128 + 1) \cdot 10 = 1\,290$
Total		$206\,922$

What percentage of the parameters is in the two convolutional layers?

$$4\,800 / 206\,922 \approx 2.3\%$$

Two observations:

- This CNN has *more* parameters than the MLP $784 \to 128 \to 10$ (101 770), yet it is more accurate on MNIST (around 99% vs. 98%) and much more robust to shifted digits. What matters is not only how many parameters a model has but how they are *arranged*.
- Almost all the parameters sit in the first dense layer, which looks at the flattened $32 \times 7 \times 7$ maps. The convolutions, which do the actual feature extraction, are cheap.

3.1 Global Average Pooling

The expensive dense layer can be removed. **Global average pooling** (GAP) averages each channel over its *entire* spatial extent, turning a (C, H, W) tensor into a vector of C numbers, one per feature. A single linear layer then maps these C features to the logits. Most modern architectures end this way. In PyTorch it is `nn.AdaptiveAvgPool2d(1)` followed by `nn.Flatten()`.

Exercise 5 Replace the dense head with GAP

(a) Apply GAP to a $(2, 2, 2)$ tensor with two channels:

$$\left(\begin{pmatrix}1 & 3\\ 0 & 4\end{pmatrix}, \begin{pmatrix}2 & 2\\ 6 & 2\end{pmatrix}\right) \xrightarrow{\text{GAP}} (2,\ 3)$$

(b) In the MNIST model, replace `Flatten` and both linear layers by GAP and one `nn.Linear(32, 10)`. The head now has $(32 + 1) \cdot 10 = 330$ parameters, and the whole network has $160 + 4\,640 + 330 = 5\,130$.

4 Backpropagation Through Pooling

Training a CNN means backpropagating through all three stages. Section 4.2 handled the convolution; ReLU passes the gradient where its input was positive and blocks it elsewhere, as in Section 3.4. Pooling has no parameters, but the gradient must still pass through it to reach the earlier layers.

- **Max pooling.** Only the largest value in each window affected the output, so the whole gradient goes to that position and the others in the window get 0.
- **Average pooling.** Every value in a $k \times k$ window contributed $1/k^2$ of the output, so each receives $1/k^2$ of the gradient.

Exercise 6 Route the gradient

The max-pooling output of Exercise 2 receives the gradient $\partial J/\partial(\text{output}) = \begin{pmatrix}0.5 & -1\\ 2 & 0.3\end{pmatrix}$. Fill in $\partial J/\partial \mathbf{H}$, using $\mathbf{H}$ from Figure 2.

$$\frac{\partial J}{\partial \mathbf{H}} = \begin{pmatrix}0 & 0 & -1 & 0\\ 0 & 0.5 & 0 & 0\\ 0 & 0 & 0.3 & 0\\ 2 & 0 & 0 & 0\end{pmatrix}$$

5 What Does a CNN Learn?

When we visualise what makes the units of a trained CNN respond, a clear hierarchy appears (Zeiler and Fergus, 2014):

- **early layers** respond to edges, colour blobs and simple textures, much like the kernels of Section 4.2;
- **middle layers** respond to combinations of these: corners, circles, grids, fur, wheels;
- **late layers** respond to object parts and whole objects: eyes, faces, car fronts.

This is the hierarchy predicted in Section 4.1: each layer builds on the features of the one below, over an ever larger receptive field.

Try it. The interactive *CNN Explainer* (`https://poloclub.github.io/cnn-explainer/`) runs a small CNN in the browser and shows every convolution, ReLU and pooling step on real images.

6 Summary

Layer	Parameters	Effect on shape (C, H, W)
Conv2d	$(C_{\text{in}}k^2 + 1)\, C_{\text{out}}$	$C \to C_{\text{out}}$; H, W by formula
ReLU	0	unchanged
MaxPool2d (2)	0	H, W halved; C unchanged
Global avg pool	0	$(C, H, W) \to (C)$
Flatten	0	$(C, H, W) \to (CHW)$
Linear	$(n_{\text{in}} + 1)\, n_{\text{out}}$	$n_{\text{in}} \to n_{\text{out}}$

- A CNN layer has three stages: convolution, ReLU, pooling.
- Max pooling keeps the largest value in each window. It has no parameters, works per channel, and makes the network approximately invariant to small shifts.
- A classification CNN is a feature extractor (conv blocks that shrink space and grow channels) followed by a classifier head (flatten or GAP, then linear layers, softmax and cross-entropy).
- In backpropagation, max pooling sends the gradient to the position of the maximum; average pooling spreads it evenly.

Coming up. Section 4.4 looks at the architectures that made CNNs famous, from LeNet-5 to ResNet, and at the design ideas each one introduced.